

Project 2 Q&A; Reference Guide

SentinelGuard Anomaly Detection System

Project 2: SentinelGuard – Anomaly Detection System

Q1: Explain the overall system design and architecture of SentinelGuard.

Hinglish Explanation (WhatsApp Language):

*SentinelGuard ek time-series anomaly detection system hai jo stock pricing returns (S&P 500) par work karta hai. Ingestion pipeline Yahoo Finance se prices download karke log-returns nikalti hai. Is sequence data ko model training ke liye prepare karte hain. Humne ek **LSTM Autoencoder** model train kiya hai normal stock behavior par. Jab model to test data diya jata hai, aur original sequence aur reconstructed sequence ke beech ka error ek dynamic threshold se bada ho jata hai, tab system use anomaly flag karta hai.*

Formal Answer (English):

SentinelGuard is an unsupervised time-series anomaly detection system optimized for stock index returns.

- **Data Processing:** Downloads historical S&P 500 data (**yfinance**), computes stationary log-returns, and builds sliding window sequences of length 10.
- **Model Training:** An LSTM Autoencoder model compiles a compressed latent representation of the data and is trained exclusively on normal (low-variance) historical periods.
- **Inference Pipeline:** Compares testing sequences with model reconstructions. The Mean Squared Error (MSE) is passed to a post-processing module that determines anomalous status based on dynamic thresholding.

New Tech / Key Concept:

- **Unsupervised Anomaly Detection:** Training a model only on normal data so it fails to reconstruct abnormal inputs, using the reconstruction error as the anomaly indicator.

Q2: How does the training process prevent the Autoencoder from learning to reconstruct anomalous sequences?

Hinglish Explanation (WhatsApp Language):

Agar hum training data mein bad stock crashes and spikes ko clean nahi karenge, toh Autoencoder model un anomalies ko reconstruct karna seekh lega. Aur testing ke time model anomalies ko perfectly reconstruct kar dega, jisse reconstruction error low aayega aur crashes flag nahi ho payenge. Isko bachane ke liye humne sequences load karte time high-volatility events ko variance calculations ke base par drop kiya hai. Hum sirf bottom 80% cleanest, low-variance sequences par model train karte hain.

Formal Answer (English):

Autoencoders detect anomalies by failing to reconstruct abnormal data patterns. If anomalous periods are present during training, the network will generalize to reconstruct them, reducing reconstruction error during testing and failing to flag anomalies.

- **Training Data Filter:** We implement a variance-based clean filter in `load_data()` inside `lstm_autoencoder.py`:

```
# 1. Filter out known anomalies (y_train == 0)
normal_idx = np.where(y_train == 0)[0]
X_normal = X_train[normal_idx]

# 2. Drop high-volatility sequences (keep bottom 80%)
seq_var = np.var(X_normal, axis=(1, 2))
var_threshold = np.percentile(seq_var, 80)
X_clean = X_normal[seq_var < var_threshold]
```

This restricts the model to learning only stable baseline market behavior.

New Tech / Key Concept:

- **Variance Pruning:** Removing high-variance records from the training partition to establish a clean normal baseline.

Q3: Explain the layer configuration of the LSTM Autoencoder and the roles of **RepeatVector** and **TimeDistributed**.

Hinglish Explanation (WhatsApp Language):

LSTM Autoencoder mein do basic sections hote hain: Encoder aur Decoder.

* **Encoder:** Input shape (10, 1) ko compress karke flat 16 dimension dense bottleneck vector banata hai.

* **Decoder:** Bottleneck representation se original sequence reconstruct karta hai.

*Bottleneck representation single vector hota hai (1D). Recurrent decoder ko timeline inputs देने के लिये **RepeatVector(10)** vector ko 10 times copy karta hai shape (10, 16) mein. Fir standard LSTM execution के बाद, **TimeDistributed(Dense(1))** हर step को independently original feature space (1 dimension) पर project karta hai.*

Formal Answer (English):

The LSTM Autoencoder in `lstm_autoencoder.py` is configured as follows:

- **Encoder:** Composed of two LSTM layers (64 and 32 units). The final LSTM layer has `return_sequences=False` to drop the temporal dimension, which is followed by a bottleneck `Dense(16, activation="tanh")` layer.

- **RepeatVector(SEQUENCE_LENGTH=10)**: Replicates the flat 1D bottleneck representation of size 16 across 10 temporal steps, producing a shape of **(10, 16)** suitable for recurrent decoder layers.
- **Decoder**: Composed of two LSTM layers (32 and 64 units) with **return_sequences=True** to preserve the time dimension.
- **TimeDistributed(Dense(N_FEATURES=1))**: Applies a dense layer weight matrix to each time step independently, mapping the shape from **(10, 64)** back to the input dimension of **(10, 1)**.

New Tech / Key Concept:

- **TimeDistributed**: A Keras wrapper that applies a Dense layer to every time step of a 3D sequence independently.

Q4: Detail the anomaly detection post-processing logic (Noise Floor, Dynamic Thresholding, and Temporal Smoothing).

Hinglish Explanation (WhatsApp Language):

Stock data mein noise bahot hoti hai, jisse model false alerts de sakta hai. Isko handle karne ke liye 3 steps use kiye hain:

1. **Noise Floor**: Bottom 70% errors ko hum exactly zero set kar dete hain, takki small fluctuations normal ho sakein.
2. **Dynamic Threshold**: Rolling window (size 60) par 85th percentile error limit compute karte hain. Static boundary ki jagah adaptive line market trends follow karti hai.
3. **Temporal Smoothing**: Agar koi alert isolated spike hai, toh use ignore karo. Anomaly tabhi validate hogi jab pichle 4 points mein se 3 points dynamic threshold se bada error dikhayenge.

Formal Answer (English):

To mitigate false positives from market noise, reconstruction errors are passed through a three-stage post-processing logic:

1. **Noise Floor Suppression**: Sets the bottom 70th percentile of errors to exactly **0.0**, eliminating minor background variance.
2. **Dynamic Thresholding**: Computes the 85th percentile of errors in a rolling window of size 60. This adapts to current market volatility levels.
3. **Temporal Smoothing**: Requires at least 75% of the last 4 consecutive sequence steps (**$0.75 * 4 = 3$** steps) to exceed the dynamic threshold before confirming a final anomaly flag.

New Tech / Key Concept:

- **Rolling Percentile Threshold**: An adaptive thresholding method that scales limits dynamically with time-series volatility trends.

Q5: Compare the LSTM Autoencoder architecture and the GRU Autoencoder. Why is GRU used as a secondary model?

Hinglish Explanation (WhatsApp Language):

Humne dono models (LSTM aur GRU) implement kiye hain.

* **GRU (Gated Recurrent Unit)**: Isme reset aur update gates hote hain. Model fast train hota hai aur operations ke liye resource footprint kam chahiye hota hai.

* **LSTM**: Isme cell state (SC_t) and forget gates hote hain. LSTM sequential details aur long-term market volatility trends ko seekhne mein GRU se better accuracy return karta hai, isliye LSTM ko primary architecture rakha hai aur GRU secondary hai.

Formal Answer (English):

- **Architectural Equivalence**: Both networks are structured symmetrically with 64 -> 32 encoding nodes, a bottleneck representation of 16, a **RepeatVector(10)** step layer, and 32 -> 64 decoding nodes.
- **GRU (Gated Recurrent Unit)**: Merges forget and input gates into a single update gate. It trains faster and requires less memory/compute resources.
- **LSTM (Long Short-Term Memory)**: Retains a separate Cell State (SC_t) coupled with input, forget, and output gates. This allows it to preserve and reconstruct long-term temporal dependencies and volatility trends in financial log-returns more effectively than the simpler GRU state.

New Tech / Key Concept:

- **GRU (Gated Recurrent Unit)**: A lightweight variant of LSTM that achieves similar sequence-modeling performance with fewer parameters.

Q6: Explain how sequence generation in **sequence_builder.py** is configured to prevent training data leakage.

Hinglish Explanation (WhatsApp Language):

Recurrent networks mein time ordering critical hoti hai. Leakage se bachane ke liye sequence design isolated rakha hai:

1. *Shuffling strictly avoid ki hai takki timeline patterns maintain rahein.*
2. *Train-test split (70/30) chronologically timeline cut points par apply kiya jata hai (matlab past 70% data training ke liye aur future 30% test data evaluation ke liye).*
3. *MinMax scaler parameter settings sirf train dataset se fit ki jati hain takki future high values training scaling leak na karein.*

Formal Answer (English):

Lookahead leakage happens when future information is utilized to scale or predict past targets. In **sequence_builder.py**, leakage is prevented through:

1. **Chronological Splitting:** Shuffling is disabled. The S&P 500 dataset is split sequentially (70% train, 30% test) by timestamp.
2. **Preprocessing Isolation:** The **MinMaxScaler** is fitted solely on the training partition. The resulting parameters are used to transform the testing data, ensuring testing boundary peaks are not leaked to training operations.
3. **Target Label Separation:** Train partition filtering is performed strictly independently of the test partition labels.

New Tech / Key Concept:

- **Chronological Split:** Splitting time-series datasets sequentially to ensure testing mirrors actual future deployment conditions.

Q7: How is the reconstruction error threshold calculated dynamically using the rolling percentile method?

Hinglish Explanation (WhatsApp Language):

Ek constant static line lagane se check fail ho jayega kyunki badalte market conditions mein base fluctuations adjust hote rehte hain. Humne python code mein step loops lagakar ek window boundary design ki hai jo har step par dynamic line update karti hai. Window size 60 steps ka percentile calculations adjust karta hai.

Formal Answer (English):

A static reconstruction threshold fails because time-series datasets experience varying phases of baseline volatility.

- **Mechanism:** The dynamic threshold calculates the 85th percentile of reconstruction errors over a historical rolling window (size=60).

```
def dynamic_threshold(errors, percentile=85, window=60):
    thresholds = np.zeros(len(errors))
    for i in range(len(errors)):
        start = max(0, i - window)
        thresholds[i] = np.percentile(errors[start:i + 1], percentile)
    return thresholds
```

This enables the system to adjust dynamically, suppressing alerts during highly volatile normal regimes.

New Tech / Key Concept:

- **Dynamic Percentile Estimation:** Adjusting anomaly limits based on rolling data distributions instead of static bounds.

Q8: What is the role of log-returns instead of raw stock prices for data preprocessing?

Hinglish Explanation (WhatsApp Language):

S&P 500 stock prices non-stationary hote hain (unka mean time ke sath change hota rehta hai jo deep learning ke liye kharab hai). Agar hum absolute prices feed karenge toh model return features detect nahi kar payega. Isliye hum prices ko log-returns mein convert karte hain: $\log(p_t / p_{t-1})$. Isse variance stable (stationary) ho jata hai aur parameters standard scale mein range karte hain.

Formal Answer (English):

Financial asset prices are non-stationary, meaning their mean and variance change over time, violating machine learning assumptions.

- **Transformation:** Log-returns are computed as:

$$r_t = \ln\left(\frac{p_t}{p_{t-1}}\right)$$

This yields a stationary time-series (stable mean and variance over time) that represents changes in price independently of the absolute index price, allowing the LSTM to capture return dynamics efficiently.

New Tech / Key Concept:

- **Stationarity:** A property of time-series where statistical parameters remain constant over time, which is essential for stable ML convergence.